Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

(CUCEI) Programación de sistemas avanzados

2026A

Carrera: Ingeniería en Computación

ENTREGA FINAL

Equipo: Pyrrinoids

Repositorio: https://github.com/ZeusdG3/SistemasAvanzados

Integrantes:

VALERIO COMPARAN JUAN RAMON

GARCIA TOMAS CARLOS ANTONIO

CORONA PLASCENCIA ZEUS HADES

- Introducción

Para el proyecto, se ejecuta mediante una VM con sistema operativo Linux (Ubuntu server) un mini-lab que emula el clúster por 1 coordinador y 4 contenedores (workers) a través de una red virtual y condiciones de red controlada mediante tc netem (latencia, pérdida, ancho de banda). Las evidencias se reflejan tanto para local, como para la VPN, en el repositorio se encuentran las instrucciones de manera clara y concisa.

- Objetivo general del proyecto

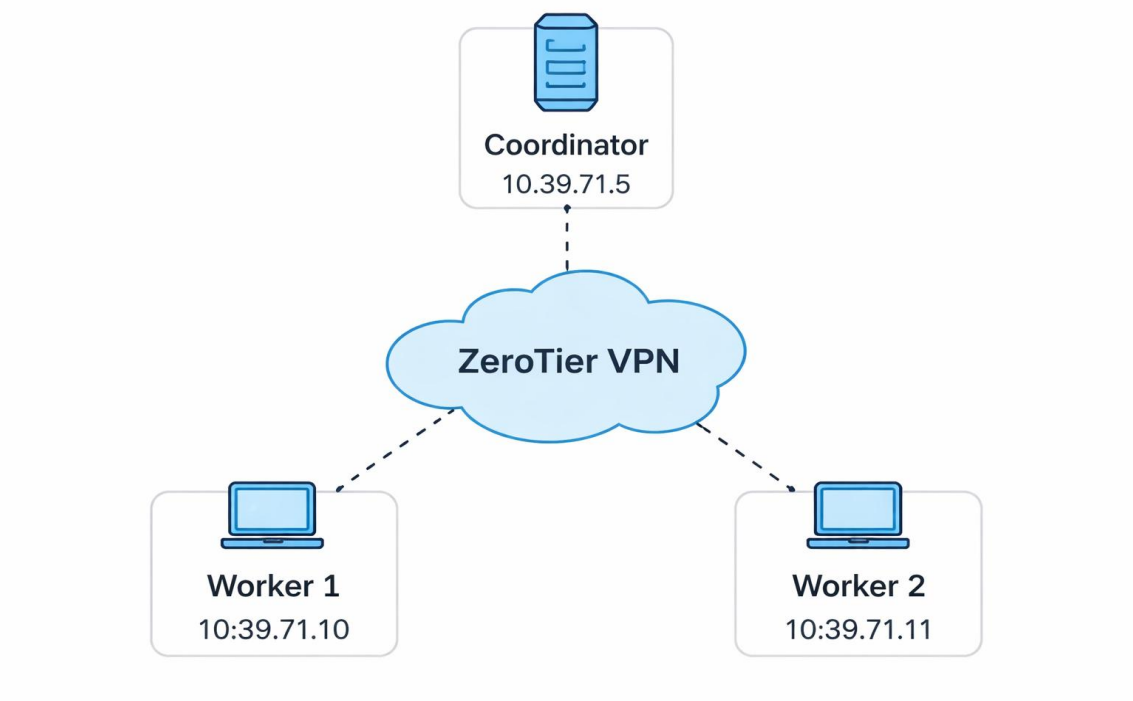
Diseñar, configurar y validar la infraestructura básica de contenedores Docker que se utilizará posteriormente para ejecutar un problema de forma distribuida.

- Desarrollo

Parte del Documento A

Diagrama final de la topología VPN

Hub and spoke, centraliza las operaciones en el nodo principal (HUB) conectado a múltiples periféricos



Descripción del rol de cada nodo.

RED VPN: 10.165.168.0/24

HUB: 10.165.168.1

Peer1: 10.165.168.153

Peer2: 10.165.168.177

Nodo 1 – HUB (10.10.10.1)

Rol: Servidor VPN

Actúa como servidor central de WireGuard

Administra las conexiones de los peers

Mantiene la tabla de peers autorizados

Permite la comunicación entre los nodos del cliente

Nodo 2 - Peer 1 (10.10.10.2)

Rol: Nodo Worker

Cliente de la VPN

Se conecta al HUB

Ejecuta contenedores Docker

Nodo 3 - Peer 2 (10.10.10.3)

Rol: Nodo Worker

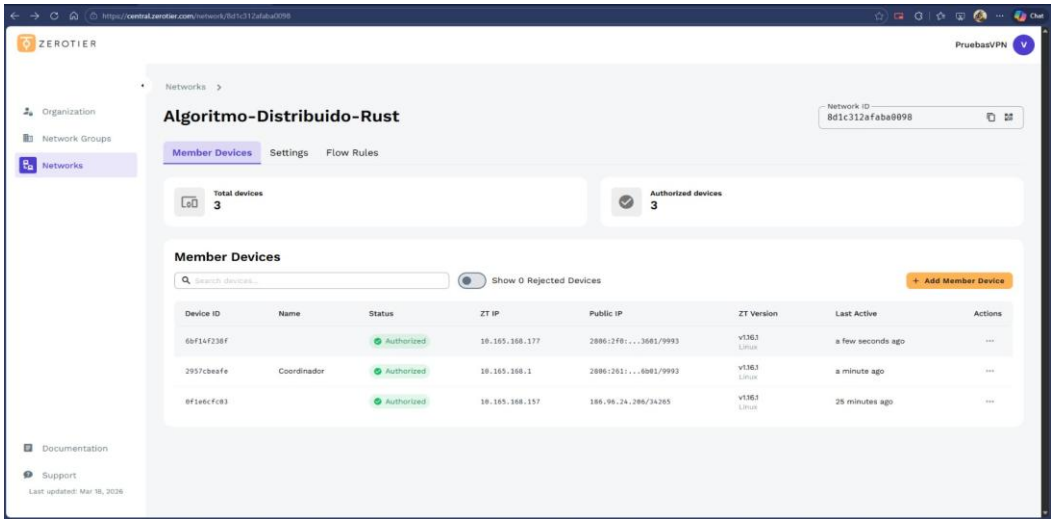
Cliente de la VPN

Se conecta al HUB

Ejecuta contenedores Docker

Evidencia de generación de la VPN con ZeroTier

Evidencia de la red VPN



Evidencia de conectividad dentro de la red

```
hades@hades:~/uvpn/output$ sudo zerotier-cli listnetworks
200 listnetworks <nwid> <name> <mac> <status> <type> <dev> <ZT assigned ips>
200 listnetworks 8d1c312afaba0098 Algoritmo-Distribuido-Rust 9a:0f:a4:96:d6:32 OK PRIVATE ztu7tiwktc 10.165.168.157/24
hades@hades:~/uvpn/output$ ping 10.165.168.1
PING 10.165.168.1 (10.165.168.1) 56(84) bytes of data.
64 bytes from 10.165.168.1: icmp_seq=1 ttl=64 time=392 ms
64 bytes from 10.165.168.1: icmp_seq=2 ttl=64 time=94.0 ms
64 bytes from 10.165.168.1: icmp_seq=3 ttl=64 time=77.6 ms
64 bytes from 10.165.168.1: icmp_seq=4 ttl=64 time=85.2 ms
64 bytes from 10.165.168.1: icmp_seq=5 ttl=64 time=80.3 ms
64 bytes from 10.165.168.1: icmp_seq=6 ttl=64 time=78.0 ms
64 bytes from 10.165.168.1: icmp_seq=7 ttl=64 time=79.9 ms
64 bytes from 10.165.168.1: icmp_seq=8 ttl=64 time=77.6 ms
64 bytes from 10.165.168.1: icmp_seq=9 ttl=64 time=78.1 ms
64 bytes from 10.165.168.1: icmp_seq=10 ttl=64 time=80.8 ms
^C
--- 10.165.168.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9005ms
rtt min/avg/max/mdev = 77.563/112.353/392.167/93.393 ms
hades@hades:~/uvpn/output$
```

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ sudo zerotier-cli listnetworks
200 listnetworks <nwid> <name> <mac> <status> <type> <dev> <ZT assigned ips>
200 listnetworks 8d1c312afaba0098 Algoritmo-Distribuido-Rust 9a:29:ed:31:c0:cf OK PRIVATE ztu7tiwktc 10.165.168.1/24
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ _
```

Evidencia de instalación de Docker y Docker Compose.

docker-compose build

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ docker-compose build_
```

Después de ejecutar el comando y al finalizar el proceso nos saldrá lo siguiente:

```
 ---> 8aee3e1476fa
Successfully built 8aee3e1476fa
Successfully tagged proyecto_sistemas_avanzados_worker4:latest
```

Para levantar los contenedores que acabamos de crear es con el siguiente comando:

docker-compose -f docker/docker-compse.worker.yml up scale worker=4

Nos saldrá lo siguiente:

```
hades@hades:~/uvpn$ docker-compose -f docker/docker-compose.worker.yml up --scale worker=4
Creating network "docker_default" with the default driver
Creating docker_worker_1 ... done
Creating docker_worker_2 ... done
Creating docker_worker_3 ... done
Creating docker_worker_4 ... done
Attaching to docker_worker_1, docker_worker_4, docker_worker_2, docker_worker_3
```

Primero, antes que nada, se ejecutará un script donde dará valores de red a los diferentes workers, como se ve de la siguiente manera:

```
1 #!/bin/bash
2 set -e
3
4 echo "=== Entrypoint: Directorio actual = $(pwd) ===="
5 echo "Contenido de /usr/src/app:"
6 ls -la /usr/src/app
7
8 # Verificar que el binario existe
9 if [ ! -f /usr/src/app/algoritmo_distribuido ]; then
10     echo "ERROR: Binario /usr/src/app/algoritmo_distribuido no encontrado"
11     exit 1
12 fi
13
14 # Aplicar reglas tc solo si WORKER_ID está definido y no vacío
15 if [ -n "$WORKER_ID" ]; then
16     echo "Aplicando reglas tc para worker $WORKER_ID"
17     sleep 2
18     case "$WORKER_ID" in
19         1) tc qdisc add dev eth0 root netem delay 120ms 30ms loss 2% rate 20Mbit ;;
20         2) tc qdisc add dev eth0 root netem delay 60ms 10ms rate 50Mbit ;;
21         3) tc qdisc add dev eth0 root netem delay 30ms 5ms loss 1% rate 30Mbit ;;
22         4) tc qdisc add dev eth0 root netem delay 80ms 40ms rate 10Mbit ;;
23         *) tc qdisc add dev eth0 root netem delay 50ms 10ms rate 100Mbit ;;
24     esac
25     echo "Reglas tc aplicadas:"
26     tc qdisc show dev eth0
27 else
28     echo "No se aplican reglas tc (WORKER_ID no definido)"
29 fi
30
31 echo "Ejecutando: $@"
32 exec "$@"
```

Aquí primero se verifica que el binario este bien, para después aplicar las reglas de tc solo si el worker está levantado, como podemos ver aquí están los diferentes parámetros dados a los workers:

```
case "$WORKER_ID" in
    1) tc qdisc add dev eth0 root netem delay 120ms 30ms loss 2% rate 20Mbit ;;
    2) tc qdisc add dev eth0 root netem delay 60ms 10ms rate 50Mbit ;;
    3) tc qdisc add dev eth0 root netem delay 30ms 5ms loss 1% rate 30Mbit ;;
    4) tc qdisc add dev eth0 root netem delay 80ms 40ms rate 10Mbit ;;
    *) tc qdisc add dev eth0 root netem delay 50ms 10ms rate 100Mbit ;;
esac
echo "Reglas tc aplicadas:"
tc qdisc show dev eth0
```

Al primero le damos un delay de entre 120ms a 20ms con una pérdida de 2% de paquete y un bando de ancha de 20Mbit.

Y así respectivamente a cada worker, y podemos sacar la conclusion de que el worker con peor conexion es el worker 1, y el mejor worker para trabajar es el worker 2.

Se uso iperf3 para probar la conectividad entre lo nodo, dandonos una de las principales dificultades para la realizacion de la interconexión de los workes en la VPN

```
Connecting to host 10.165.168.1, port 5201
[  5] local 10.165.168.157 port 36610 connected to 10.165.168.1 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  1.38 MBytes  11.5 Mbits/sec   32    131 KBytes
[  5]   1.00-2.00   sec  1.25 MBytes  10.5 Mbits/sec   10    107 KBytes
[  5]   2.00-3.00   sec  1.25 MBytes  10.5 Mbits/sec    2   85.9 KBytes
[  5]   3.00-4.00   sec   896 KBytes  7.34 Mbits/sec    4   72.5 KBytes
[  5]   4.00-5.00   sec   640 KBytes  5.24 Mbits/sec    2   67.1 KBytes
[  5]   5.00-6.00   sec   512 KBytes  4.19 Mbits/sec   13   24.2 KBytes
[  5]   6.00-7.00   sec   384 KBytes  3.15 Mbits/sec    4   24.2 KBytes
[  5]   7.00-8.00   sec   384 KBytes  3.15 Mbits/sec    0   40.3 KBytes
[  5]   8.00-9.00   sec   512 KBytes  4.19 Mbits/sec    0   56.4 KBytes
[  5]   9.00-10.00  sec   896 KBytes  7.34 Mbits/sec    0   75.1 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  8.00 MBytes  6.71 Mbits/sec   67             sender
[  5]   0.00-10.09  sec  7.62 MBytes  6.34 Mbits/sec                  receiver

iperf Done.
```

- Captura del docker-compose.yml base.

```yaml
version: '3.8'
version: "3.8"

services:
  coordinator:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    volumes:
      - ./output:/output
    environment:
      - RUST_LOG=info

  worker1:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    environment:
      - WORKER_ID=1
    depends_on:
      - coordinator

  worker2:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    environment:
      - WORKER_ID=2
    depends_on:
      - coordinator

  worker3:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    environment:
      - WORKER_ID=3
    depends_on:
      - coordinator

  worker4:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    environment:
      - WORKER_ID=4
    depends_on:
      - coordinator

networks:
  distnet:
    driver: bridge
```

Definición y justificación de del problema computacional distribuido a implementar

División del dominio: el coordinador divide la imagen en regiones o bloques (por ejemplo, filas o tiles).
Asignación a workers: cada worker recibe una subregión para calcular los valores de Mandelbrot.
Cálculo iterativo: cada píxel se itera según la fórmula zn+1=zn2+cz_{n+1} = z_n^2 + czn+1=zn2+c hasta alcanzar el máximo de iteraciones o escapar del conjunto.
Retorno de resultados: los workers devuelven la subimagen calculada al coordinador.
Composición final: el coordinador combina todas las subregiones para generar la imágen completa.
Tolerancia a fallos: si un worker falla o hay retrasos por la red, el coordinador asigna la tarea a otro worker.

Funcionamiento del programa

Bien una vez explicado esto, lo que nos saldrá al levantar los contenedores será lo siguiente en la terminal, primero veremos que el coordinador se levanta y ejecuta el programa de Rust, imprime en pantalla los parámetros que asignamos en este como la región a renderizar, iteraciones y el supersampling.

```
coordinator_1  | Contenido de /usr/src/app:
coordinator_1  | total 2572
coordinator_1  | drwxr-xr-x 1 root root    4096 Mar 18 02:10 .
coordinator_1  | drwxr-xr-x 1 root root    4096 Mar  9 00:44 ..
coordinator_1  | -rwxr-xr-x 1 root root 2624816 Mar 18 02:10 algoritmo_distribuido
coordinator_1  | No se aplican reglas tc (WORKER_ID no definido)
coordinator_1  | Ejecutando: ./algoritmo_distribuido
coordinator_1  | Coordinador escuchando en 0.0.0.0:8080 (Docker: coordinator:8080)
coordinator_1  | Región a renderizar:
coordinator_1  |   x: [-0.554999, -0.254999]
coordinator_1  |   y: [-0.674707, -0.505957]
coordinator_1  |   Iteraciones: 500
coordinator_1  |   Supersampling: 2x2
```

Después los worker iniciaran uno por uno

```
worker2_1       | === Entrypoint: Directorio actual = /usr/src/app ===
worker2_1       | Contenido de /usr/src/app:
worker2_1       | total 2572
worker2_1       | drwxr-xr-x 1 root root    4096 Mar 18 02:10 .
worker2_1       | drwxr-xr-x 1 root root    4096 Mar  9 00:44 ..
worker2_1       | -rwxr-xr-x 1 root root 2624816 Mar 18 02:10 algoritmo_distribuido
worker1_1       | === Entrypoint: Directorio actual = /usr/src/app ===
worker1_1       | Contenido de /usr/src/app:
worker2_1       | Aplicando reglas tc para worker 2
worker4_1       | === Entrypoint: Directorio actual = /usr/src/app ===
worker4_1       | Contenido de /usr/src/app:
worker1_1       | total 2572
worker1_1       | drwxr-xr-x 1 root root    4096 Mar 18 02:10 .
worker1_1       | drwxr-xr-x 1 root root    4096 Mar  9 00:44 ..
worker1_1       | -rwxr-xr-x 1 root root 2624816 Mar 18 02:10 algoritmo_distribuido
worker4_1       | total 2572
worker4_1       | drwxr-xr-x 1 root root    4096 Mar 18 02:10 .
worker4_1       | drwxr-xr-x 1 root root    4096 Mar  9 00:44 ..
worker4_1       | -rwxr-xr-x 1 root root 2624816 Mar 18 02:10 algoritmo_distribuido
worker1_1       | Aplicando reglas tc para worker 1
worker4_1       | Aplicando reglas tc para worker 4
worker3_1       | === Entrypoint: Directorio actual = /usr/src/app ===
worker3_1       | Contenido de /usr/src/app:
worker3_1       | total 2572
worker3_1       | drwxr-xr-x 1 root root    4096 Mar 18 02:10 .
worker3_1       | drwxr-xr-x 1 root root    4096 Mar  9 00:44 ..
worker3_1       | -rwxr-xr-x 1 root root 2624816 Mar 18 02:10 algoritmo_distribuido
worker3_1       | Aplicando reglas tc para worker 3
```

Podemos ver que a cada worker se le aplican las reglas anteriormente mencionadas y después están listos para recibir tareas.

```
worker4_1       | Reglas tc aplicadas:
worker4_1       | qdisc netem 804c: root refcnt 5 limit 1000 delay 80ms  40ms rate 10Mbit
worker4_1       | Ejecutando: ./algoritmo_distribuido worker
worker3_1       | Reglas tc aplicadas:
worker3_1       | qdisc netem 804d: root refcnt 5 limit 1000 delay 30ms  5ms loss 1% rate 30Mbit
worker3_1       | Ejecutando: ./algoritmo_distribuido worker
worker1_1       | Reglas tc aplicadas:
worker1_1       | qdisc netem 804e: root refcnt 5 limit 1000 delay 120ms  30ms loss 2% rate 20Mbit
worker1_1       | Ejecutando: ./algoritmo_distribuido worker
worker2_1       | Reglas tc aplicadas:
worker2_1       | qdisc netem 804f: root refcnt 5 limit 1000 delay 60ms  10ms rate 50Mbit
worker2_1       | Ejecutando: ./algoritmo_distribuido worker
```

Y con esto tendremos los contenedores levantados en nuestra máquina, cabe mencionar que como el sistema se ejecuta en cada máquina localmente, después de ejecutar se iniciara el programa en Rust, donde veremos a continuación que es lo que pasa a detalle.

```
worker3_1       | Worker conectado al coordinador
coordinator_1   | Worker conectado desde: 172.18.0.4:42442
coordinator_1   | Enviando tarea 15 (filas 4050..4320)
worker3_1       | Procesando tarea 15 (filas 4050..4320)
worker2_1       | Worker conectado al coordinador
worker4_1       | Worker conectado al coordinador
coordinator_1   | Worker conectado desde: 172.18.0.6:60780
coordinator_1   | Enviando tarea 14 (filas 3780..4050)
worker2_1       | Procesando tarea 14 (filas 3780..4050)
coordinator_1   | Worker conectado desde: 172.18.0.3:49448
worker4_1       | Procesando tarea 13 (filas 3510..3780)
coordinator_1   | Enviando tarea 13 (filas 3510..3780)
worker1_1       | Worker conectado al coordinador
coordinator_1   | Worker conectado desde: 172.18.0.5:36262
coordinator_1   | Enviando tarea 12 (filas 3240..3510)
worker1_1       | Procesando tarea 12 (filas 3240..3510)
```

Aquí el coordinador empieza a recibir que los workers están conectados con él y les envía una tarea.

```
worker3_1       | Tarea 15 completada en 9.315807943s
worker4_1       | Tarea 14 completada en 9.330239351s
worker2_1       | Tarea 13 completada en 9.554222014s
worker1_1       | Tarea 12 completada en 9.613275331s
worker2_1       | Tarea 13 enviada en 3.118892865s
```

Aqui como vemos, los workers empiezan a terminar sus tareas y envían un mensaje.

```
coordinator_1   | Resultado recibido de tarea 13 (filas 3510..3780)
coordinator_1   | Progreso: 1/16 tareas completadas, tiempo transcurrido: 20.548344724s
coordinator_1   | Enviando tarea 11 (filas 2970..3240)
worker2_1       | Procesando tarea 11 (filas 2970..3240)
worker4_1       | Tarea 14 enviada en 9.462625556s
coordinator_1   | Resultado recibido de tarea 14 (filas 3780..4050)
coordinator_1   | Progreso: 2/16 tareas completadas, tiempo transcurrido: 28.510870446s
coordinator_1   | Enviando tarea 10 (filas 2700..2970)
```

Y aquí vemos que los workers empiezan a enviar los resultados en el tiempo permitido, pero algunos, como el worker1 tendrán muchos problemas de conexión para enviar el resultado en tan sólo 30 segundos.

```
worker1_1       | Timeout enviando resultado de tarea 12: deadline has elapsed
worker1_1       | Error en worker: Timeout enviando resultado
worker4_1       | Tarea 10 enviada en 9.584637964s
worker1_1       | Worker conectado al coordinador
```

Entonces, así pasará hasta que el coordinador reciba todas las tareas y empiece a armar la imagen, como podemos ver, que en este intento, después de 124 segundos, osea 2 min. terminaron de armar la imagen, el coordinador guarda la imagen en /output/ como mandelbrot.png

```
worker1_1       | Worker sin tareas, esperando y reconectando...
worker3_1       | Coordinador indica no más tareas. Terminando.
worker3_1       | Worker sin tareas, esperando y reconectando...
coordinator_1   | Resultado recibido de tarea 5 (filas 1350..1620)
coordinator_1   | Progreso: 16/16 tareas completadas, tiempo transcurrido: 124.014787197s
coordinator_1   | Imagen guardada como '/output/mandelbrot.png'
coordinator_1   | Imagen guardada. Terminando coordinador...
```

Después se desconectará, y los workers también.

```
proyecto_sistemas_avanzados_worker2_1 exited with code 0
worker4_1       | Error en worker: failed to lookup address information: Temporary failure in name resolution
worker4_1       | Demasiados errores de conexión. Worker termina.
proyecto_sistemas_avanzados_worker4_1 exited with code 0
worker3_1       | Error en worker: failed to lookup address information: Temporary failure in name resolution
worker1_1       | Error en worker: failed to lookup address information: Temporary failure in name resolution
worker1_1       | Demasiados errores de conexión. Worker termina.
proyecto_sistemas_avanzados_worker1_1 exited with code 0
worker3_1       | Error en worker: failed to lookup address information: Temporary failure in name resolution
worker3_1       | Demasiados errores de conexión. Worker termina.
proyecto_sistemas_avanzados_worker3_1 exited with code 0
```

Vista del coordinador

```
Ubuntu server-24.04 en LACAPRICHOSA: Conexión a máquina virtual
Archivo  Acción  Medios  Portapapeles  Ver  Ayuda
coordinator     | Resultado recibido de tarea 15 (filas 4050..4320)
coordinator     | Progreso: 1/16 tareas completadas, tiempo transcurrido: 4.77026987s
coordinator     | Enviando tarea 11 (filas 2970..3240)
coordinator     | Resultado recibido de tarea 14 (filas 3780..4050)
coordinator     | Progreso: 2/16 tareas completadas, tiempo transcurrido: 5.901639088s
coordinator     | Enviando tarea 10 (filas 2700..2970)
coordinator     | Resultado recibido de tarea 13 (filas 3510..3780)
coordinator     | Progreso: 3/16 tareas completadas, tiempo transcurrido: 6.25633864s
coordinator     | Enviando tarea 9 (filas 2430..2700)
coordinator     | Resultado recibido de tarea 12 (filas 3240..3510)
coordinator     | Progreso: 4/16 tareas completadas, tiempo transcurrido: 6.313787048s
coordinator     | Enviando tarea 8 (filas 2160..2430)
coordinator     | Resultado recibido de tarea 11 (filas 2970..3240)
coordinator     | Progreso: 5/16 tareas completadas, tiempo transcurrido: 9.831478736s
coordinator     | Enviando tarea 7 (filas 1890..2160)
coordinator     | Resultado recibido de tarea 8 (filas 2160..2430)
coordinator     | Progreso: 6/16 tareas completadas, tiempo transcurrido: 11.886470316s
coordinator     | Enviando tarea 6 (filas 1620..1890)
coordinator     | Resultado recibido de tarea 10 (filas 2700..2970)
coordinator     | Progreso: 7/16 tareas completadas, tiempo transcurrido: 12.107434666s
coordinator     | Enviando tarea 5 (filas 1350..1620)
coordinator     | Resultado recibido de tarea 9 (filas 2430..2700)
coordinator     | Progreso: 8/16 tareas completadas, tiempo transcurrido: 13.083279667s
coordinator     | Enviando tarea 4 (filas 1080..1350)
coordinator     | Resultado recibido de tarea 7 (filas 1890..2160)
coordinator     | Progreso: 9/16 tareas completadas, tiempo transcurrido: 14.7464404s
coordinator     | Enviando tarea 3 (filas 810..1080)
coordinator     | Resultado recibido de tarea 5 (filas 1350..1620)
coordinator     | Progreso: 10/16 tareas completadas, tiempo transcurrido: 18.07134012s
coordinator     | Enviando tarea 2 (filas 540..810)
coordinator     | Resultado recibido de tarea 6 (filas 1620..1890)
coordinator     | Progreso: 11/16 tareas completadas, tiempo transcurrido: 18.612223982s
coordinator     | Enviando tarea 1 (filas 270..540)
coordinator     | Resultado recibido de tarea 3 (filas 810..1080)
coordinator     | Progreso: 12/16 tareas completadas, tiempo transcurrido: 20.58725349s
coordinator     | Enviando tarea 0 (filas 0..270)
coordinator     | Resultado recibido de tarea 4 (filas 1080..1350)
coordinator     | Progreso: 13/16 tareas completadas, tiempo transcurrido: 20.646135329s
coordinator     | Resultado recibido de tarea 0 (filas 0..270)
coordinator     | Progreso: 14/16 tareas completadas, tiempo transcurrido: 25.846515063s
coordinator     | Worker conectado desde: 10.165.168.177:45400
coordinator     | Resultado recibido de tarea 2 (filas 540..810)
coordinator     | Progreso: 15/16 tareas completadas, tiempo transcurrido: 28.885855817s
coordinator     | Resultado recibido de tarea 1 (filas 270..540)
coordinator     | Progreso: 16/16 tareas completadas, tiempo transcurrido: 30.14538903s
coordinator     | Imagen guardada como '/output/mandelbrot.png'
coordinator     | Imagen guardada. Terminando coordinador...
juan@lacaprichosa:~/proyecto_sistemas_avanzados$
Estado: ejecutando
```

Ubuntu server-24.04 en LACAPRICHOSA: Conexión a máquina virtual

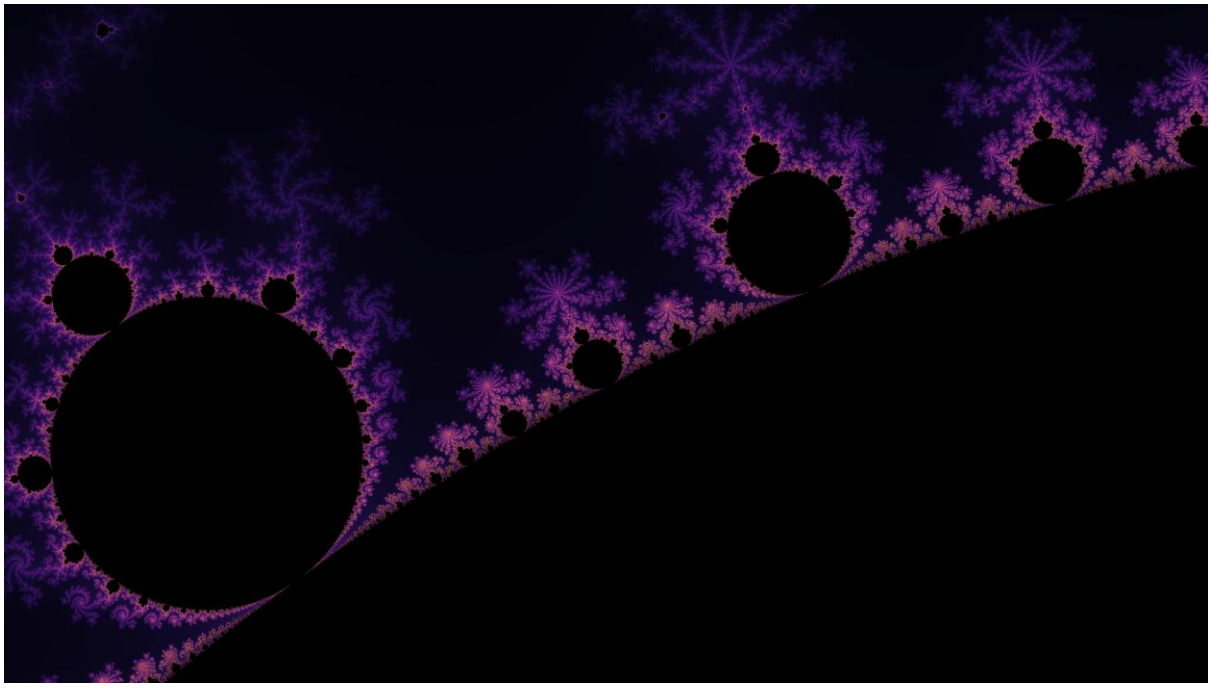
Archivo Acción Medios Portapapeles Ver Ayuda

```
coordinator     | Resultado recibido de tarea 13 (filas 3510..3780)
coordinator     | Progreso: 3/16 tareas completadas, tiempo transcurrido: 174.205907003s
coordinator     | Enviando tarea 9 (filas 2430..2700)
coordinator     | Resultado recibido de tarea 12 (filas 3240..3510)
coordinator     | Progreso: 4/16 tareas completadas, tiempo transcurrido: 174.895294407s
coordinator     | Enviando tarea 8 (filas 2160..2430)


^[
coordinator     | Resultado recibido de tarea 8 (filas 2160..2430)
coordinator     | Progreso: 5/16 tareas completadas, tiempo transcurrido: 319.536765812s
coordinator     | Enviando tarea 7 (filas 1890..2160)
coordinator     | Resultado recibido de tarea 10 (filas 2700..2970)
coordinator     | Progreso: 6/16 tareas completadas, tiempo transcurrido: 323.320630315s
coordinator     | Enviando tarea 6 (filas 1620..1890)
coordinator     | Resultado recibido de tarea 11 (filas 2970..3240)
coordinator     | Progreso: 7/16 tareas completadas, tiempo transcurrido: 324.01694776s
coordinator     | Enviando tarea 5 (filas 1350..1620)
coordinator     | Resultado recibido de tarea 9 (filas 2430..2700)
coordinator     | Progreso: 8/16 tareas completadas, tiempo transcurrido: 329.420750564s
coordinator     | Enviando tarea 4 (filas 1080..1350)
coordinator     | Resultado recibido de tarea 4 (filas 1080..1350)
coordinator     | Progreso: 9/16 tareas completadas, tiempo transcurrido: 450.980614517s
coordinator     | Enviando tarea 3 (filas 810..1080)
coordinator     | Resultado recibido de tarea 5 (filas 1350..1620)
coordinator     | Progreso: 10/16 tareas completadas, tiempo transcurrido: 453.694888376s
coordinator     | Enviando tarea 2 (filas 540..810)
coordinator     | Resultado recibido de tarea 6 (filas 1620..1890)
coordinator     | Progreso: 11/16 tareas completadas, tiempo transcurrido: 457.275941102s
coordinator     | Enviando tarea 1 (filas 270..540)
coordinator     | Resultado recibido de tarea 7 (filas 1890..2160)
coordinator     | Progreso: 12/16 tareas completadas, tiempo transcurrido: 460.134394236s
coordinator     | Enviando tarea 0 (filas 0..270)
coordinator     | Resultado recibido de tarea 0 (filas 0..270)
coordinator     | Progreso: 13/16 tareas completadas, tiempo transcurrido: 561.52322468s
coordinator     | Resultado recibido de tarea 1 (filas 270..540)
coordinator     | Progreso: 14/16 tareas completadas, tiempo transcurrido: 562.300520639s
coordinator     | Resultado recibido de tarea 2 (filas 540..810)
coordinator     | Progreso: 15/16 tareas completadas, tiempo transcurrido: 566.534332286s
coordinator     | Worker conectado desde: 10.165.168.177:52080
coordinator     | Worker conectado desde: 10.165.168.177:45942
coordinator     | Resultado recibido de tarea 3 (filas 810..1080)
coordinator     | Progreso: 16/16 tareas completadas, tiempo transcurrido: 569.291662461s
coordinator     | Imagen guardada como '/output/mandelbrot.png'
coordinator     | Imagen guardada. Terminando coordinador...
juan@lacaprichosa:~/proyecto_sistemas_avanzados$
juan@lacaprichosa:~/proyecto_sistemas_avanzados$
juan@lacaprichosa:~/proyecto_sistemas_avanzados$
```

Estado: ejecutando

Resultado



Descripción de problemas encontrados y decisiones técnicas tomadas.

Curva de aprendizaje en linux sin entorno gráfico.

Hemos tenido que visualizar los comandos que son necesarios para poder realizar la práctica de manera correspondiente, es muy importante verificar las configuraciones ya que al no contar con entorno gráfico cualquier cambio aplicado solamente se puede acceder mediante comandos.

Configuración de Wireguard.

Nosotros hemos encontrado incidencias a la hora de hacer ping entre nodos para poder tener conexión dentro de la VPN, hemos configurado correctamente los archivos wg0 de manera correspondiente, teniendo en cuenta el Port Forwarding del modem, así como no contar con CGNAT y contar con una IP pública que permita poder ser el HUB.

Por lo que se terminó cambiando el proyecto con la creacion de la plataforma ZeroTier con el fin de facilitar la conexión entre casas

Docker.

Se tuvieron que hacer algunos cambios ya que se ejecutaban tanto de manera local, como por conexión en la Red.

Parte del Documento B

Programa distribuido en Rust (algoritmo de mandelbrot)

A continuación, se muestra el algoritmo distribuido de mandelbrot en Rust, está diseñado para que el contenedor principal sea un coordinador que reparte tareas con coordenadas para el armado del algoritmo, asimismo, contenedores siendo workers que son los que procesan los datos para después regresarlos al coordinador y que el arme la imagen completa.

El programa está diseñado para cambiar y modificar los parámetros a gusto, así como la resolución de la imagen, el máximo de iteraciones, y por último supersampling, para mejorar la resolución de píxeles.

Ahora pasaremos a explicar el código:

Coordinator.rs

```rust
use crate::messages::{Task, ResultMsg};
use tokio::net::{TcpListener, TcpStream};
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
use tokio::time::{timeout, Duration};
use std::sync::Arc;
use tokio::sync::Mutex;
use std::collections::HashMap;
use image::{ImageBuffer, Rgb};
use std::sync::atomic::{AtomicBool, Ordering};
use std::process;

// Parámetros globales
const WIDTH: u32 = 7680;
const HEIGHT: u32 = 4320;
const MAX_ITER: u32 = 500;
const SUPERSAMPLING: u32 = 2;

// Región a renderizar
const CENTER_RE: f64 = -0.4049987;
const CENTER_IM: f64 = -0.5903320;
const ZOOM: f64 = 10.0;
const FULL_WIDTH: f64 = 3.0;
```

En esta primera parte, se puede ver cómo se importan las librerías a usar, así como las funciones de los demás archivos.

Primero definimos los parámetros globales, como la resolución (que está en 8k), las máximas iteraciones, y si usamos supersampling.

Después definimos los valores de la región a renderizar.

```rust
pub async fn run_coordinator() -> tokio::io::Result<()> {
    let listener = TcpListener::bind("0.0.0.0:8080").await?;
    println!("Coordinador escuchando en 0.0.0.0:8080 (Docker: coordinator:8080)");

    // Calcular los límites de la región
    let width = FULL_WIDTH / ZOOM;
    let height = width * (HEIGHT as f64 / WIDTH as f64);
    let x_min = CENTER_RE - width / 2.0;
    let x_max = CENTER_RE + width / 2.0;
    let y_min = CENTER_IM - height / 2.0;
    let y_max = CENTER_IM + height / 2.0;

    println!("Región a renderizar:");
    println!("  x: [{:.6}, {:.6}]", x_min, x_max);
    println!("  y: [{:.6}, {:.6}]", y_min, y_max);
    println!("  Iteraciones: {}", MAX_ITER);
    println!("  Supersampling: {}x{}", SUPERSAMPLING, SUPERSAMPLING);
```

Aquí la funcion run_coordinator() es la que inicia al coordinador, primero lo ponemos a escuchar todas las conexiones.

Luego hacemos el cálculo de los límites de la región con los valores anteriormente definidos, imprimimos en pantalla el resultado de lo que se trabajara, y al final creamos tareas, para repartirlas con los workers, aquí pusimos como ejemplo 16 tareas.

```rust
// Crear tareas: dividir la imagen en bandas horizontales
let num_tasks = 16;
let band_height = HEIGHT / num_tasks;
let mut tasks = Vec::new();
for i in 0..num_tasks {
    let y_start = i * band_height;
    let y_end = if i == num_tasks - 1 { HEIGHT } else { (i + 1) * band_height };
    tasks.push(Task {
        id: i,
        x_min,
        x_max,
        y_min,
        y_max,
        width: WIDTH,
        global_height: HEIGHT,
        y_start,
        y_end,
        max_iter: MAX_ITER,
        supersampling: SUPERSAMPLING,
    });
}
```

Después definimos las variables para su manejo distribuido de las tareas,

```rust
let tasks_arc = Arc::new(Mutex::new(tasks));
let results_arc = Arc::new(Mutex::new(HashMap::new()));
let total_tasks = num_tasks;
let assembled_flag = Arc::new(AtomicBool::new(false));
```

También una variable para medir el tiempo antes de que trabajen los workers, y la parte donde recibimos si los workers están conectados a nosotros,

```
let start_time = std::time::Instant::now();

loop {
    let (socket, addr) = listener.accept().await?;
    println!("Worker conectado desde: {:?}", addr);

    let tasks = tasks_arc.clone();
    let results = results_arc.clone();
    let flag = assembled_flag.clone();
    let start = start_time.clone();

    tokio::spawn(async move {
        if let Err(e) = handle_worker(socket, tasks, results, total_tasks as usize, flag, start).await {
            eprintln!("Error manejando worker: {}", e);
        }
    });
```

Después definimos la función donde manejamos a los workers, pasamos los parámetros anteriores

```
async fn handle_worker(
    mut socket: TcpStream,
    tasks: Arc<Mutex<Vec<Task>>>,
    results: Arc<Mutex<HashMap<u32, (u32, u32, Vec<u8>)>>>,
    total_tasks: usize,
    assembled_flag: Arc<AtomicBool>,
    start_time: std::time::Instant,
) -> tokio::io::Result<()> {
```

Si no hay más tareas en el array de tasks mandamos el siguiente mensaje:

```
let task = match task_opt {
    Some(t) => t,
    None => {
        writer.write_all(b"NO_MORE_TASKS\n").await?;
        writer.flush().await?;
        break;
    }
};
```

Y si hay tareas, las enviamos, también agregamos un timeout por si la tarea no es enviada en máximo 10 segundos, así verificamos que la conexión es correcta.

```
println!("Enviando tarea {} (filas {}..{})", task.id, task.y_start, task.y_end);

let serialized = serde_json::to_string(&task).unwrap() + "\n";
if let Err(_) = timeout(Duration::from_secs(10), writer.write_all(serialized.as_bytes())).await {
    eprintln!("Timeout enviando tarea {}, reinsertando", task.id);
    tasks.lock().await.push(task);
    continue;
}
writer.flush().await?;
```

Después recibimos la tarea en un máximo de 4 min, esto está pensado por si son tareas pesadas, y si el resultado recibido esta mal, mandamos un error y regresamos al array la tarea

```
line.clear();
// Timeout de 240 segundos (4 minutos) para tareas muy pesadas
let read_result = timeout(Duration::from_secs(240), reader.read_line(&mut line)).await;
match read_result {
    Ok(Ok(n)) if n > 0 => {
        let result: ResultMsg = match serde_json::from_str(&line) {
            Ok(r) => r,
            Err(e) => {
                eprintln!("Error parseando resultado de tarea {}: {}", task.id, e);
                tasks.lock().await.push(task);
                continue;
            }
        };
```

```
println!("Resultado recibido de tarea {} (filas {}..{})", result.id, result.y_start, result.y_end);
{
    let mut results_map = results.lock().await;
    results_map.insert(result.id, (result.y_start, result.y_end, result.data));
    let completed = results_map.len();
    let elapsed = start_time.elapsed();
    println!("Progreso: {}/{} tareas completadas, tiempo transcurrido: {:?}", completed, total_tasks, elapsed);
```

Aquí si no hay error, decimos que recibimos la tarea y la marcamos como completa y mandamos un mensaje con el progreso total que llevamos.

```
if completed == total_tasks && !assembled_flag.load(Ordering::Relaxed) {
    if assembled_flag.compare_exchange(false, true, Ordering::SeqCst, Ordering::SeqCst).is_ok() {
        let results = results.clone();
        tokio::spawn(async move {
            if let Err(e) = assemble_image(results, WIDTH, HEIGHT).await {
                eprintln!("Error ensamblando imagen: {}", e);
            } else {
                println!("Imagen guardada. Terminando coordinador...");
                process::exit(0);
            }
        });
```

Después si todas las tareas fueron completadas, se arma la imagen en la función assemble_image y si no hay error, se crea y guarda en la direccion /output/

```
async fn assemble_image(
    results: Arc<Mutex<HashMap<u32, (u32, u32, Vec<u8>)>>>,
    width: u32,
    height: u32,
) -> tokio::io::Result<()> {
```

```rust
let results_map = results.lock().await;
let mut img = ImageBuffer::<Rgb<u8>, Vec<u8>>::new(width, height);

for (_id, (y_start, y_end, data)) in results_map.iter() {
    let y_start = *y_start;
    let y_end = *y_end;
    let rows = (y_end - y_start) as usize;
    let row_bytes = (width * 3) as usize;
    assert_eq!(data.len(), rows * row_bytes, "Tamaño de datos incorrecto");

    for (i, y) in (y_start..y_end).enumerate() {
        let row_start = i * row_bytes;
        let row_data = &data[row_start..row_start + row_bytes];
        for x in 0..width as usize {
            let r = row_data[x * 3];
            let g = row_data[x * 3 + 1];
            let b = row_data[x * 3 + 2];
            img.put_pixel(x as u32, y, Rgb([r, g, b]));
        }
    }
}
```

```rust
img.save("/output/mandelbrot.png").map_err(|e| {
    eprintln!("Error guardando imagen: {}", e);
    tokio::io::Error::new(tokio::io::ErrorKind::Other, e)
})?;S_
println!("Imagen guardada como '/output/mandelbrot.png'");
```

Worker.rs

```rust
use crate::messages::{Task, ResultMsg};
use tokio::net::TcpStream;
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
use tokio::time::{timeout, Duration};
use image::Rgb;
```

```rust
pub async fn run_worker() -> tokio::io::Result<()> {
    // Esperar unos segundos a que el coordinador esté listo
    tokio::time::sleep(Duration::from_secs(5)).await;

    let mut connect_errors = 0;
    const MAX_CONNECT_ERRORS: u32 = 5;
```

Aquí se espera 5 segundos para que el coordinador esté listo, solo por precaución, si hay más de 5 errores de conexión, daremos por hecho que no podemos conectarnos y apagamos el worker

```rust
loop {
    match connect_and_work().await {
        Ok(()) => {
            // connect_and_work retorna Ok cuando recibe NO_MORE_TASKS o cierre normal
            println!("Worker sin tareas, esperando y reconectando...");
            tokio::time::sleep(Duration::from_secs(5)).await;
            connect_errors = 0;
        }
        Err(e) => {
            eprintln!("Error en worker: {}", e);
            connect_errors += 1;
            if connect_errors >= MAX_CONNECT_ERRORS {
                eprintln!("Demasiados errores de conexión. Worker termina.");
                break;
            }
            tokio::time::sleep(Duration::from_secs(2)).await;
        }
    }
}
Ok(())
```

Después se define la funcion connect_and_work

```rust
36 async fn connect_and_work() -> tokio::io::Result<()> {
37     let mut stream = TcpStream::connect("coordinator:8080").await?;
38     println!("Worker conectado al coordinador");
39
40     let (reader, mut writer) = stream.split();
41     let mut reader = BufReader::new(reader);
42     let mut line = String::new();
```

```rust
44     loop {
45         line.clear();
46         let read_result = timeout(Duration::from_secs(30), reader.read_line(&mut line)).await;
47         let line = match read_result {
48             Ok(Ok(n)) if n > 0 => line.trim().to_string(),
49             Ok(Ok(_)) => {
50                 println!("Coordinador cerró la conexión");
51                 return Ok(());
52             }
53             Ok(Err(e)) => return Err(e),
54             Err(_) => {
55                 eprintln!("Timeout esperando tarea");
56                 return Err(tokio::io::Error::new(tokio::io::ErrorKind::TimedOut, "Timeout"));
57             }
58         };
```

Hacemos un bucle para verificar que todo esté correcto, que no haya desconexiones del coordinador o si hay un timeout por esperar tarea.

```rust
60             if line == "NO_MORE_TASKS" {
61                 println!("Coordinador indica no más tareas. Terminando.");
62                 return Ok(());
63             }
```

Si recibimos el NO_MORE_TASKS cerramos sesión con el worker.

```rust
        let task: Task = match serde_json::from_str(&line) {
            Ok(t) => t,
            Err(e) => {
                eprintln!("Error parseando tarea: {}", e);
                continue;
            }
        };
```

También manejamos errores parseando tareas.

```rust
        println!("Procesando tarea {} (filas {}..{})", task.id, task.y_start, task.y_end);

        // Medir tiempo de cómputo
        let compute_start = std::time::Instant::now();
        let data = compute_mandelbrot_chunk(&task);
        let compute_time = compute_start.elapsed();
        println!("Tarea {} completada en {:?}", task.id, compute_time);
```

Después se manda a imprimir la tarea que estamos procesando, y si es completada, mandamos a imprimir el tiempo de cómputo total de la tarea.

```rust
    let result = ResultMsg {
        id: task.id,
        y_start: task.y_start,
        y_end: task.y_end,
        data,
    };
```

Guardamos el resultado

```rust
        let serialized = serde_json::to_string(&result).unwrap() + "\n";

        // Medir tiempo de envío
        let send_start = std::time::Instant::now();

        // Bloque de envío (con timeout de 30)
        if let Err(e) = timeout(Duration::from_secs(30), writer.write_all(serialized.as_bytes())).await {
            eprintln!("Timeout enviando resultado de tarea {}: {}", task.id, e);
            return Err(tokio::io::Error::new(tokio::io::ErrorKind::TimedOut, "Timeout enviando resultado"));
        }
        writer.flush().await?;

        let send_time = send_start.elapsed();
        println!("Tarea {} enviada en {:?}", task.id, send_time);
    }
}
```

Serializamos y mandamos la tarea al coordinador, si hay un timeout de 30 segundos como máximo al enviarla, se cancelara y se enviara un error, para que esa tarea pueda ser reorganizada por el coordinador, si todo sale bien, se manda un mensaje diciendo que la tarea fue enviada con éxito en el tiempo total que fue tomado.

Después se definen las funciones donde se procesa el algoritmo de mandelbrot y se iteran los colores.

```rust
fn compute_mandelbrot_chunk(task: &Task) -> Vec<u8> {
    let width = task.width;
    let height = task.global_height;
    let y_start = task.y_start;
    let y_end = task.y_end;
    let supersampling = task.supersampling;
    let max_iter = task.max_iter;
    let x_min = task.x_min;
    let x_max = task.x_max;
    let y_min = task.y_min;
    let y_max = task.y_max;

```

```rust
    let mut data = Vec::with_capacity(((y_end - y_start) * width) as usize * 3);

    for y in y_start..y_end {
        for x in 0..width {
            let mut r_sum = 0u32;
            let mut g_sum = 0u32;
            let mut b_sum = 0u32;
            let samples = supersampling * supersampling;

            for sy in 0..supersampling {
                for sx in 0..supersampling {
                    let sub_x = (sx as f64 + 0.5) / supersampling as f64;
                    let sub_y = (sy as f64 + 0.5) / supersampling as f64;

                    let c_re = x_min + ((x as f64 + sub_x) / width as f64) * (x_max - x_min);
                    let c_im = y_min + ((y as f64 + sub_y) / height as f64) * (y_max - y_min);

                    let iter = mandelbrot(c_re, c_im, max_iter);
                    let Rgb([r, g, b]) = iter_to_color(iter, max_iter);
                    r_sum += r as u32;
                    g_sum += g as u32;
                    b_sum += b as u32;
                }
            }

            data.push((r_sum / samples) as u8);
            data.push((g_sum / samples) as u8);
            data.push((b_sum / samples) as u8);
        }
    }
    data
}
```

```rust
fn mandelbrot(c_re: f64, c_im: f64, max_iter: u32) -> u32 {
    let mut z_re = 0.0;
    let mut z_im = 0.0;
    let mut iter = 0;

    while iter < max_iter {
        let z_re_new = z_re * z_re - z_im * z_im + c_re;
        let z_im_new = 2.0 * z_re * z_im + c_im;

        z_re = z_re_new;
        z_im = z_im_new;

        if z_re * z_re + z_im * z_im > 4.0 {
            break;
        }
        iter += 1;
    }
    iter
}

fn iter_to_color(iter: u32, max_iter: u32) -> Rgb<u8> {
    if iter == max_iter {
        Rgb([0, 0, 0])
    } else {
        let t = iter as f64 / max_iter as f64;
        let grad = colorgrad::magma();
        let [r, g, b, _] = grad.at(t).to_rgba8();
        Rgb([r, g, b])
    }
}
```

Messages.rs

Aquí se definen los valores de Task y ResultMsg

```rust
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct Task {
    pub id: u32,
    pub x_min: f64,
    pub x_max: f64,
    pub y_min: f64,
    pub y_max: f64,
    pub width: u32,
    pub global_height: u32,
    pub y_start: u32,
    pub y_end: u32,
    pub max_iter: u32,
    pub supersampling: u32,
}

#[derive(Serialize, Deserialize, Debug)]
pub struct ResultMsg {
    pub id: u32,
    pub y_start: u32,
    pub y_end: u32,
    pub data: Vec<u8>,
}
```

Creacion y construccion de imagen y contenedores de docker

Primero definimos el Dockerfile y el docker-compose en el directorio de docker de nuestro proyecto.

Dockerfile

```dockerfile
# Etapa de compilación
FROM rust:latest AS builder

WORKDIR /usr/src/app

# Copiar todo el contenido de la carpeta rust (Cargo.toml y src/)
COPY rust/ .

# Compilar en modo release
RUN cargo build --release

# Imagen final
FROM debian:bookworm-slim

# Instalar iproute2 para tc y netcat-openbsd
RUN apt-get update && apt-get install -y iproute2 netcat-openbsd && rm -rf /var/lib/apt/lists/*

WORKDIR /usr/src/app

# Copiar el binario compilado
COPY --from=builder /usr/src/app/target/release/algoritmo_distribuido .

# Copiar el script de entrada
COPY docker/entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

ENTRYPOINT ["/entrypoint.sh"]
CMD ["./algoritmo_distribuido"]
```

docker-compose.yml

```yaml
version: '3.8'
version: "3.8"

services:
  coordinator:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    volumes:
      - ./output:/output
    environment:
      - RUST_LOG=info

  worker1:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    environment:
      - WORKER_ID=1
    depends_on:
      - coordinator

  worker2:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    environment:
      - WORKER_ID=2
    depends_on:
      - coordinator

  worker3:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    environment:
      - WORKER_ID=3
    depends_on:
      - coordinator

  worker4:
    build:
      context: .
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    networks:
      - distnet
    cap_add:
      - NET_ADMIN
    environment:
      - WORKER_ID=4
    depends_on:
      - coordinator

networks:
  distnet:
    driver: bridge
```

Añadimos unos files de docker-compose tanto para el worker como el coordinador, esto para garantizar su funcionamiento con la VPN de ZeroTier

Coordinador

```yaml
version: '3.8'

services:
  coordinator:
    image: algoritmo_distribuido
    container_name: coordinator
    network_mode: host
    volumes:
      - ./output:/output
    restart: unless-stopped
```

Worker

```yaml
version: '3.8'

services:
  worker:
    image: algoritmo_distribuido
    command: ["./algoritmo_distribuido", "worker"]
    environment:
      - COORDINATOR_ADDR=${COORDINATOR_ADDR:-10.165.168.1:8080}
    restart: unless-stopped
```

Después de definir los valores de configuración, pasamos a crear la imagen del algoritmo con el siguiente comando:

docker-compose build

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ docker-compose build_
```

```
Successfully built 61743bd6fd3f
Successfully tagged algoritmo_distribuido:latest
```

Después de ejecutar el comando y al finalizar el proceso nos saldrá lo siguiente:

```
 ---> 8aee3e1476fa
Successfully built 8aee3e1476fa
Successfully tagged proyecto_sistemas_avanzados_worker4:latest
```

Para levantar los contenedores que acabamos de crear es con el siguiente comando:

docker-compose up

Nos saldrá lo siguiente:

```
Starting proyecto_sistemas_avanzados_worker2_1 ... done
Starting proyecto_sistemas_avanzados_worker3_1 ... done
Starting proyecto_sistemas_avanzados_worker4_1 ... done
Starting proyecto_sistemas_avanzados_worker1_1 ... done
```

```
coordinator_1  | Contenido de /usr/src/app:
coordinator_1  | total 2572
coordinator_1  | drwxr-xr-x 1 root root    4096 Mar 18 02:10 .
coordinator_1  | drwxr-xr-x 1 root root    4096 Mar  9 00:44 ..
coordinator_1  | -rwxr-xr-x 1 root root 2624816 Mar 18 02:10 algoritmo_distribuido
coordinator_1  | No se aplican reglas tc (WORKER_ID no definido)
coordinator_1  | Ejecutando: ./algoritmo_distribuido
coordinator_1  | Coordinador escuchando en 0.0.0.0:8080 (Docker: coordinator:8080)
coordinator_1  | Región a renderizar:
coordinator_1  |   x: [-0.554999, -0.254999]
coordinator_1  |   y: [-0.674707, -0.505957]
coordinator_1  |   Iteraciones: 500
coordinator_1  |   Supersampling: 2x2
```

Y con esto tendremos los contenedores levantados en nuestra máquina y ejecutando el programa.

Plan de ejecución (en bullets):

Para la medición de rendimiento en el sistema distribuido se usarán las siguientes herramientas:

- **Tiempo de ejecución total:** medición con `**time**` en Linux o `**Instant**` en Rust.
- **Latencia de comunicación:** timestamps en mensajes y pruebas de red con `**ping**` o `**iperf3**`.
- **Throughput:** contador de tareas procesadas por unidad de tiempo en el coordinador.
- **Uso de recursos:** monitoreo con `**docker stats**`, `**htop**` o `**top**`.
- **Escalabilidad:** ejecución con distinto número de workers y comparación de resultados.
- **Tolerancia a fallos:** pruebas deteniendo contenedores para observar la reacción del coordinador.

Parte del Documento C

Metodología base

Comunicación activa mediante WhatsApp

Se utilizará un tipo de Scrum ligero

Sprints cortos de una semana

Reuniones mediante Google Meet

Uso de trello como herramienta de gestión (Product Backlog y Sprint)

Roles

Los 3 integrantes participan en:

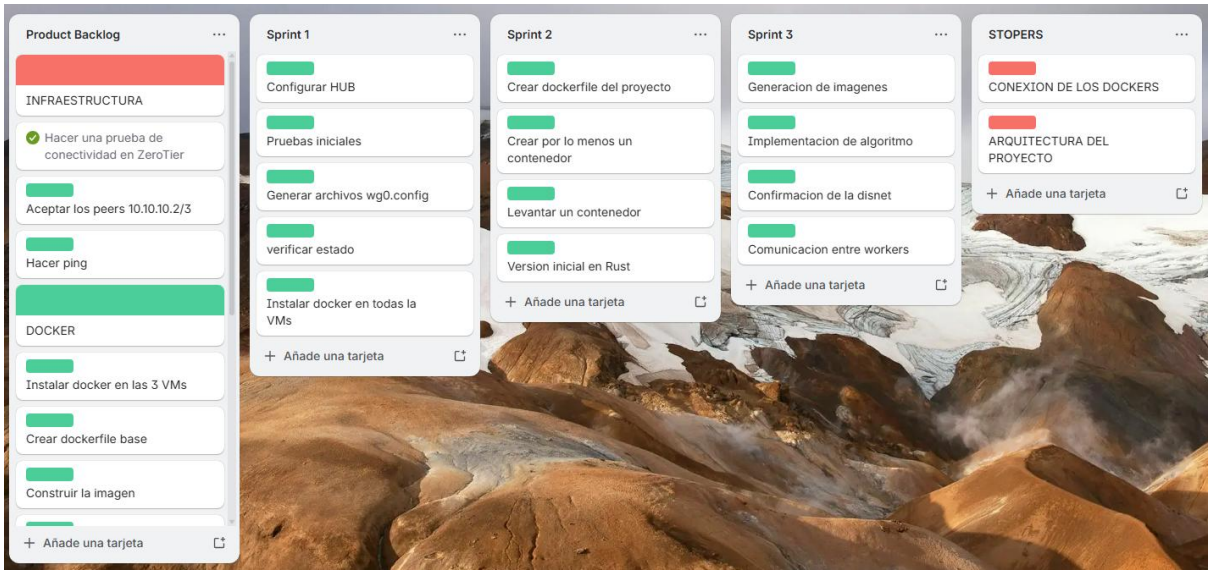
- Configuración de infraestructura
- Desarrollo del algoritmo
- Documentación y pruebas

Responsable técnico (Rotativo por sprint)

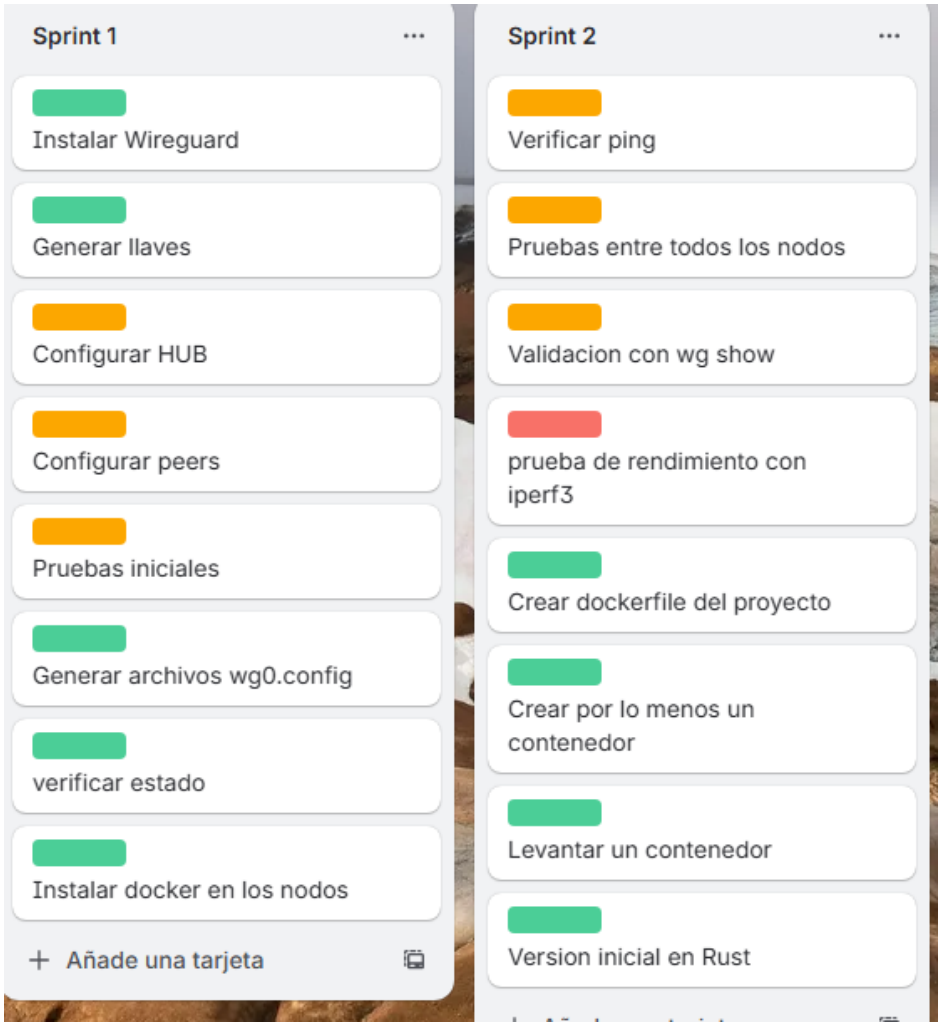
- Supervisar la configuración
- Coordinar las tareas del sprint
- Validar las pruebas

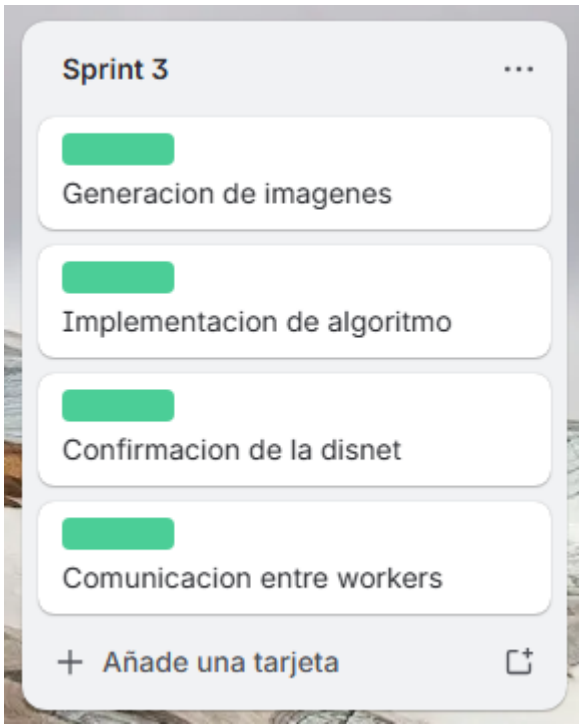
Artefactos (Evidencia presentable)

Product Backlog

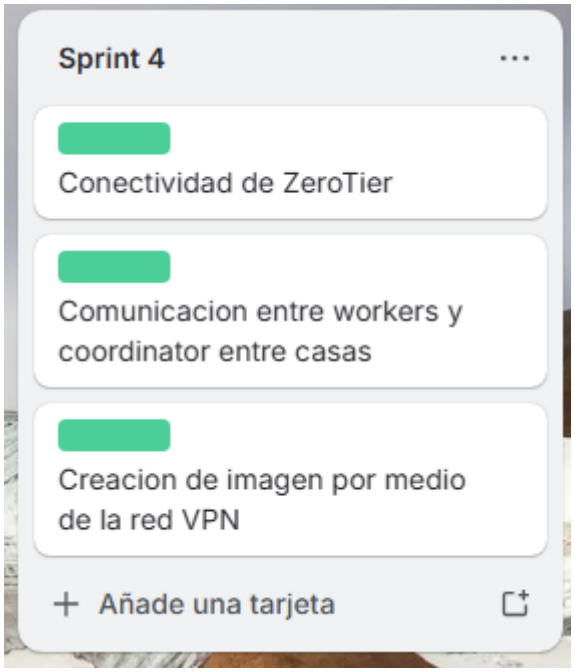

Product Backlog
INFRAESTRUCTURA
Hacer una prueba de conectividad en ZeroTier
Aceptar los peers 10.10.10.2/3
Hacer ping
DOCKER
Instalar docker en las 3 VMs
Crear dockerfile base
Construir la imagen
Añade una tarjeta
Sprint 1
Configurar HUB
Pruebas iniciales
Generar archivos wg0.config
verificar estado
Instalar docker en todas la VMs
Añade una tarjeta
Sprint 2
Crear dockerfile del proyecto
Crear por lo menos un contenedor
Levantar un contenedor
Version inicial en Rust
Añade una tarjeta
Sprint 3
Generacion de imagenes
Implementacion de algoritmo
Confirmacion de la disnet
Comunicacion entre workers
Añade una tarjeta
STOPERS
CONEXION DE LOS DOCKERS
ARQUITECTURA DEL PROYECTO
Añade una tarjeta

Sprints


Sprint 1
Instalar Wireguard
Generar llaves
Configurar HUB
Configurar peers
Pruebas iniciales
Generar archivos wg0.config
verificar estado
Instalar docker en los nodos
Añade una tarjeta
Sprint 2
Verificar ping
Pruebas entre todos los nodos
Validacion con wg show
prueba de rendimiento con iperf3
Crear dockerfile del proyecto
Crear por lo menos un contenedor
Levantar un contenedor
Version inicial en Rust


Sprint 3
Generacion de imagenes
Implementacion de algoritmo
Confirmacion de la disnet
Comunicacion entre workers
Añade una tarjeta

Y un último sprint el cual se hizo el dia de la presentación para terminar de configurar la comunicación por medio de la VPN, ya que se generó una confusión y no se realizó este procedimiento



Pruebas de minutas

A	B	C	D	E	F
Nombre	**Apellido**	**Correo electrónico**	**Duración**	**Hora a la que se unió**	**Hora a la que salió**
CARLOS ANTONIO	GARCIA TOMAS	carlos.garcia6654@alumnos.udg.mx	2 h 30 min	20:13	22:43
JUAN RAMON	VALERIO COMPARAN	juan.valerio5617@alumnos.udg.mx	2 h 31 min	20:12	22:43
ZEUS HADES	CORONA PLASCENCIA	zeus.corona0285@alumnos.udg.mx	2 h 30 min	20:13	22:43

A	B	C	D	E	F
Nombre	**Apellido**	**Correo electrónico**	**Duración**	**Hora a la que se unió**	**Hora a la que salió**
Hades	Corona	copl*********@***.com	2 h 31 min	15:58	19:02
ZEUS HADES	CORONA PLASCENCIA	zeus.corona0285@alumnos.udg.mx	2 h 52 min	15:47	19:02
CARLOS ANTONIO	GARCIA TOMAS	carlos.garcia6654@alumnos.udg.mx	3 h 19 min	15:43	19:02
JUAN RAMON	VALERIO COMPARAN	juan.valerio5617@alumnos.udg.mx	3 h 19 min	15:42	19:02

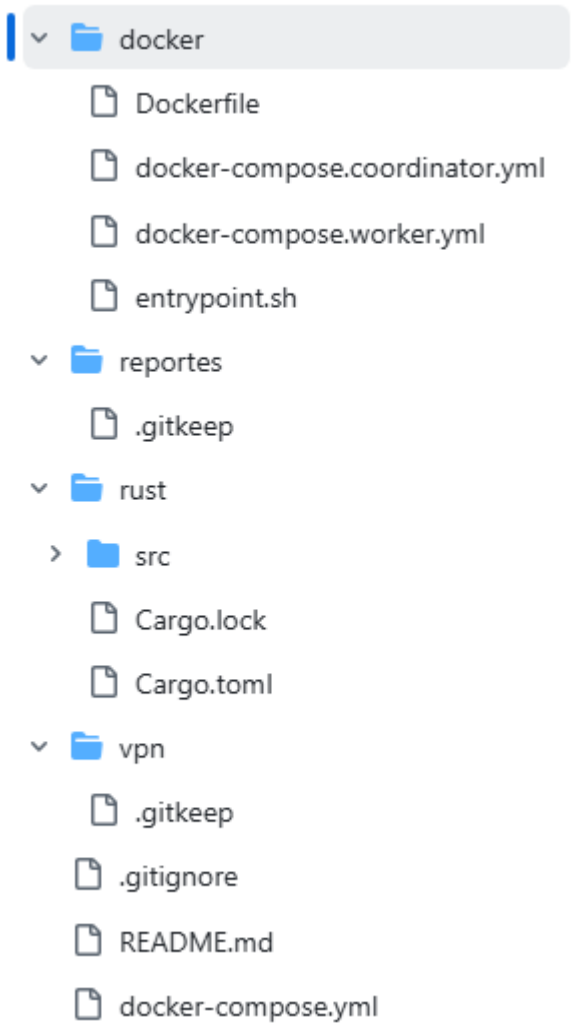
Elades inició una llamada que duró 5 horas. 7/3/2026 12:52

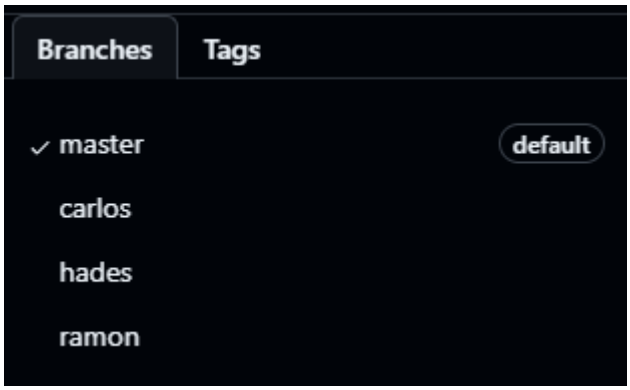
Parte del Documento D

Evidencias y documentación

README listo dentro del repositorio GitHub

Estructura del proyecto





Commits

Commits on Mar 7, 2026

Update Dockerfile 2 ZeusdG3 authored 2 weeks ago	Verified	ff5cd96
Update worker.rs ZeusdG3 authored 2 weeks ago	Verified	af73f45
Update coordinator.rs ZeusdG3 authored 2 weeks ago	Verified	87de40d
Update docker-compose.yml ZeusdG3 authored 2 weeks ago	Verified	67f75c4
Update Dockerfile ZeusdG3 authored 2 weeks ago	Verified	329ea81

Commits on Feb 18, 2026

Add initial README for distributed system in Rust JuanComparan authored last month	Verified	dbe8ba0
Agregando carpetas faltantes para la estructura JuanComparan committed last month		317266d
Organizando estructura de proyecto, docker completado JuanComparan committed last month		6241641

Commits on Mar 18, 2026

README v2 carlosgarcia6654 authored yesterday	Verified	74c4be1
Comentarios arreglados JuanComparan committed yesterday		8600e0c
Cambios en comentarios mal puestos JuanComparan committed yesterday		0ccb54a

Commits on Mar 17, 2026

Comentarios arreglados JuanComparan committed 2 days ago		0af6a7d

Commits on Mar 8, 2026

Merge pull request #1 from ZeusdG3/ramon JuanComparan authored 2 weeks ago	Verified	4094a68
Algoritmo de Mandelbrot implementado con red disnet JuanComparan committed 2 weeks ago		34eff54

Commits on Mar 19, 2026

Revise README for distributed Mandelbrot rendering project JuanComparan authored 1 hour ago	Verified	fe3a820
Update worker.rs ZeusdG3 authored 2 hours ago	Verified	a00b45c
Merge branch 'master' of https://github.com/ZeusdG3/SistemasAvanzados JuanComparan committed 3 hours ago		53e8c75
Algotimo Distribuido funcionando con VPN Zerotier JuanComparan committed 3 hours ago		83a79f8

Conclusión

En este punto hemos entendido varios puntos los cuales son vitales para que la práctica funciones, por ende, las configuraciones correspondientes de la VPN son vitales para que la red de contenedores funcione y puedan hacer de manera correcta programación paralela.

WSL2 era nuestra configuración inicial la cual presentó problemas al realizarla como HUB dentro de la configuración de Windows por el firewall y porque mantiene una IP diferente y cambiante cada que se reinicia el equipo, sin embargo, puede utilizarse como cliente ya que no es necesario mantener un enlace fijo, manejar en una máquina creada por hyperV fue la opción final e ideal para terminar el proyecto.

Se logró completar con la totalidad del proyecto a pesar de las dificultades presentadas y las opciones brindadas por el profesor, y aunque al principio solo se hizo de forma local extendiendo evidencias para la parte de la VPN como el clúster emulado se mantiene la parte importante que es la emulación de un sistema distribuido, el proyecto estaba preparado para la interconexión por VPN implementada por ZeroTier.

El Equipo por temas de organización se vio limitado de tiempo para poder realizar pruebas en conjunto, aun así, consideramos que por parte del profesor se nos fue otorgado una prórroga justa por diferentes cuestiones presentadas en el tiempo de impartición de la materia.

Por otro lado, queda recalcar que los problemas presentados al inicio, como la poca o nula experiencia con las herramientas de desarrollo entorpecieron las primeras semanas de elaboración del proyecto, planteando una situación retadora pero que se gestionó al final de este.

Referencias

Instalar Hyper-V. (s/f). Recuperado el 20 de febrero de 2026, de Microsoft.com website: https://learn.microsoft.com/es-es/windows-server/virtualization/hyper-v/get-started/install-hyper-v?tabs=powershell&pivots=windows

Get Ubuntu Server. (s/f). Recuperado el 20 de febrero de 2026, de Ubuntu website: https://ubuntu.com/download/server

Create a network. (s/f). Zerotier.com. Recuperado el 20 de marzo de 2026, de https://docs.zerotier.com/start/